HDOC DAY-70

Write a recursive function to calculate the digital root of a number. Use a static variable to count the number of recursive call.

Algorithm:

1. Start.
2. Read a positive integer n.
3. Call digitalRoot(n).
4. Inside the function, declare a **static variable count** and increment it for every function call.
5. If n < 10, print the number of recursive calls and return n.
6. Otherwise, calculate the sum of all digits of n.
7. Recursively call digitalRoot(sum).
8. Display the digital root.
9. Stop.

C Program:

```
#include <stdio.h>


int digitalRoot(int n)
{
   static int count = 0;
   count++;


   // Base case
   if (n < 10)
   {
      printf("Number of recursive calls = %d\n", count);
      return n;
   }
```

```
    // Find sum of digits
    int sum = 0;
    int temp = n;

    while (temp > 0)
    {
        sum += temp % 10;
        temp /= 10;
    }

    // Recursive call
    return digitalRoot(sum);
}

int main()
{
    int n;

    printf("Enter a positive number: ");
    scanf("%d", &n);

    printf("Digital Root = %d\n", digitalRoot(n));

    return 0;
}
```

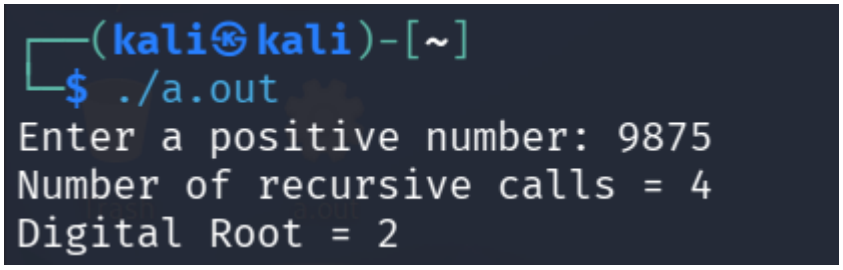
Test Case Table:

Test Case	Input	Digit Reduction	Digit Root	Recursive Calls
1	9875	9875 – 29 – 11 – 2	2	4
2	12345	12345-15-6	6	3
3	99	99-18-9	9	3
4	7	7	7	1
5	5555	5555-20-2	2	3

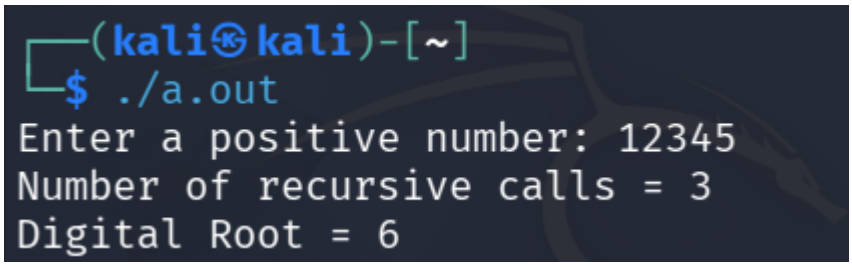
Compilation and Execution:

The following program has been compiled and executed successfully.

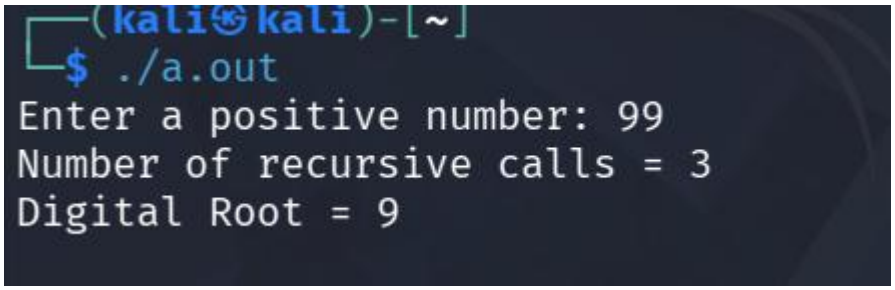
Test Case 1:



Test Case 2:



Test Case 3:



Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter a positive number: 7
Number of recursive calls = 1
Digital Root = 7
```

Test Case 5:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter a positive number: 5555
Number of recursive calls = 3
Digital Root = 2
```